

CocinaP

Necesidades del Proyecto

Sistema de Seguridad Inteligente para Cocina
Julio 2026

1. Resumen

CocinaP es un sistema de seguridad para cocina con detección de fuego, humo y cocina desatendida via IA (YOLO11n). Tiene dos componentes:

- Aplicacion Windows (Python + PySide6): Servidor local con camara, configuracion y alarmas.
- Aplicacion Movil (Flutter): Monitoreo remoto via WiFi. Android funcional, iOS pendiente.

Estado actual: Windows y Android terminados y verificados. iOS es el proximo hito.

2. Necesidades Creativas

Windows - COMPLETADO

- GUI nativa con 3 pestañas: Cámara (video + detecciones), Configuración (sliders), Alarmas (historial).
- Indicador de riesgo por colores (verde/amarillo/naranja/rojo).

Android - COMPLETADO

- Diseño Material Design 3: discovery mDNS, dashboard con video + status cards, configuración remota, historial de alarmas, servicio en foreground.

iOS - PENDIENTE

- Adaptar navegación a UITabBar nativa de iOS.
- Usar tipografía SF Pro y gestos nativos (swipe back, pull-to-refresh).
- Adaptar video a notch y Dynamic Island.
- Soporte de modo oscuro nativo.
- Iconografía con SF Symbols de Apple.

Identidad Visual - PENDIENTE

- Crear logotipo profesional (.ico, .png, SVG).
- Diseñar icono de app para Android e iOS.

3. Necesidades Tecnicas

Infraestructura

- Ya tenemos: PC Windows 11, Python 3.14, Flutter SDK, Android SDK, JDK 21, dispositivo Android fisico, GitHub.
- Hace falta: Mac con Xcode 15+ (para compilar iOS) y iPhone fisico para pruebas.

Cuentas

- Ya tenemos: Firebase (gratis) para notificaciones push.
- Hace falta: Apple Developer Program (\$99/anual) para publicar en App Store y habilitar notificaciones push (APNs).
- Opcional: Google Play Console (\$25 unico) para publicar Android en Play Store.

Software

- Backend Windows: Python + OpenCV + PySide6 + YOLO11n + zeroconf + firebase-admin. Empaquetado con PyInstaller.
- App Android: Flutter + Dart + http + provider + firebase_messaging + flutter_background_service.
- App iOS: Mismas librerias Flutter compartidas. BGTaskScheduler nativo en vez de flutter_background_service. CocoaPods + Xcode 15+.

Servidor (completado)

- API REST con 10 endpoints, stream MJPEG (downscaled 480p), eventos SSE, mDNS/Zeroconf, FCM opcional.
- No requiere cambios para iOS.

4. Riesgos

- Firewall Windows: Puerto 8080 bloqueado sin admin. Mitigacion: conexion manual por IP.
- iOS background: Apple no permite servicios persistentes. Mitigacion: FCM/APNs + BGTaskScheduler.
- Sin Mac no se puede compilar iOS. Mitigacion: MacStadium cloud (~\$50/mes) o GitHub Actions macOS.
- App Store puede rechazar por uso de red local. Mitigacion: documentar justificacion en Info.plist.

5. Proximos Pasos

Corto plazo (1-2 semanas)

- Adquirir Mac para compilacion iOS.
- Crear cuenta Apple Developer.
- Configurar proyecto Flutter para iOS.
- Distribuir APK Android a testers.

Mediano plazo (3-4 semanas)

- Implementar background task en iOS.
- Adaptar diseno a lineamientos Apple.
- Pruebas en iPhone real.
- Publicar en TestFlight.
- Crear logotipo y assets graficos.

Largo plazo (2-3 meses)

- Publicar en App Store y Google Play.
- Mejoras de rendimiento YOLO.
- Soporte para multiples camaras.